

Universidade Federal do Espírito Santo
Departamento de Informática
Trabalho Prático
Sistemas Operacionais (INF15980)

Implementação de um Mini-Kernel *Multithread* com suporte a Escalonamento por FCFS, RR e Prioridade (Preemptiva)

1a Etapa

1. Objetivo Geral

Desenvolver, em linguagem C, um escalonador de processos multithread com três políticas (FCFS, Round Robin e Prioridade Preemptiva), usando threads POSIX (pthread) para representar os fluxos de execução de cada processo. O Linux implementa o modelo 1:1 de threads: cada thread é visível ao kernel e escalonada como tal. O comportamento do sistema deve respeitar esse paradigma.

O sistema deve ser entregue em duas versões: monoprocessador e multiprocessador. Nesta última, o número de CPUs é lido do arquivo de entrada e há duas arquiteturas de fila (fila global e fila por CPU com migração). Ao final, o programa gera o log de eventos e um relatório de métricas de desempenho.

O trabalho consolida os seguintes conceitos:

- Escalonamento e sincronização de processos e threads
- Controle de concorrência com mutexes e variáveis de condição
- Estruturas de controle: BCP (Bloco de Controle de Processo) e TCB (Task Control Block)
- Políticas FCFS, RR e Prioridade Preemptiva
- Filas de execução, balanceamento de carga entre CPUs e simulação de tempo
- Avaliação de desempenho de políticas de escalonamento

A 2ª etapa estenderá este kernel com novas políticas e novos tipos de processo. Modularize o código, mantendo a política de escalonamento isolada do restante do sistema.

2. Visão Geral do Mini-Kernel

Cada processo é representado por um BCP (PID, duração, prioridade, chegada, número de threads etc.) e contém várias threads, todas sincronizadas por mutexes e variáveis de condição. O sistema deve:

- Emular a criação e a execução concorrente de processos com múltiplas threads
- Manter a(s) fila(s) de prontos e aplicar a política escolhida
- Realizar a sincronização e a preempção entre os processos e o(s) escalonador(es)
- Registrar o log de eventos e calcular as métricas

2.1 Convenções gerais

- Tempo: em milissegundos, medido com gettimeofday a partir do instante T0 em que a simulação começa (logo após a leitura do arquivo de entrada).
- FATIA = 100 ms: menor unidade de execução simulada. Duração, tempo de chegada e quantum são sempre múltiplos de 100 nos arquivos de teste.
- Prioridade: de 1 a 5, sendo 1 a mais alta e 5 a mais baixa. As threads herdam a prioridade do processo.
- PID: lido do arquivo de entrada (não confundir com getpid()). É único, mas não necessariamente sequencial nem ordenado.
- Desempates (regras obrigatórias):
    1. Processos que chegam no mesmo instante entram na fila em ordem crescente de PID.
    2. Se, no instante *t*, um processo chega e outro é reinserido por fim de quantum, o que chegou entra primeiro.
    3. Se mais de uma CPU está livre, o processo é atendido pela de menor índice.
    4. Na política de prioridade, prioridades iguais são desempatadas pela ordem de entrada na fila (FIFO).

2.2 Inicialização dos processos

O programa lê o arquivo de entrada (seção 4.1) e preenche a lista de processos. Cada BCP é inicializado com remaining_time = process_len, estado READY, mutex e variável de condição inicializados e o vetor de threads alocado.

2.3 Chegada e criação de threads

Uma thread geradora aguarda até que o tempo atual alcance o tempo de chegada de cada processo (em ordem de chegada). Usar usleep em intervalos curtos é aceitável; espera ocupada pura é desencorajada. Ao chegar, o processo tem suas threads criadas e é inserido na fila de prontos, e o evento de chegada é registrado no log.

As threads são criadas mas não trabalham imediatamente: ficam bloqueadas na variável de condição cv do processo até que o escalonador o coloque em RUNNING. Quando todos os processos já chegaram, a variável global generator_done é ativada.

2.4 Execução do processo pelas threads

Cada thread executa um loop:

1. Trava o mutex do processo e, enquanto o estado não for RUNNING (e o processo não estiver FINISHED), bloqueia-se em cv.
2. Quando liberada, simula uma fatia de execução com usleep(FATIA * 1000).
3. Desconta o tempo executado de remaining_time de forma segura (sob mutex).
4. Se remaining_time chegar a zero, a thread que detectar primeiro muda o estado para FINISHED, registra a finalização e faz broadcast para as demais encerrarem.

Contabilização de tempo. Em cada fatia, todas as threads do processo executam em paralelo, e juntas consomem exatamente 100 ms de remaining_time. A thread de índice *i* desconta FATIA / num_threads + (i < FATIA % num_threads ? 1 : 0) ms. Assim, um processo de duração *D* ocupa uma CPU por exatamente *D* ms, independentemente do número de threads.

Fronteiras de fatia. O escalonador só altera o estado de um processo em fronteiras de fatia. Uma thread conclui a fatia em andamento antes de bloquear. Sugestão: usar pthread_barrier_t para manter as threads de um processo alinhadas por fatia.

2.5 Escalonador e políticas

Cada CPU é simulada por uma thread escalonadora própria (na versão monoprocessador, há uma só). Ela monitora a fila de prontos, seleciona o próximo processo, muda seu estado para RUNNING, registra o evento, faz broadcast na cv do processo e aguarda o fim do processo ou do quantum, conforme a política.

FCFS: Sem quantum e sem preempção. O processo executa até finalizar, e a CPU é liberada nesse instante. A ordem de execução é a ordem de entrada na fila.

Round Robin: O quantum *Q* vem do arquivo de entrada. Ao ser despachado, o processo detém a CPU por *Q* ms. Se terminar antes, a finalização é registrada no instante em que remaining_time chega a zero, mas a CPU só é liberada ao fim do quantum (a CPU fica retida e ociosa, e esse tempo não conta como ocupado na utilização). Ao fim do quantum, se remaining_time > 0, o processo volta a READY, o evento *Quantum esgotado* é registrado e ele é reinserido no fim da fila.

Prioridade Preemptiva: Usa o mesmo mecanismo de quantum e retenção da CPU do RR. A regra de preempção é única e explícita: a preempção só ocorre ao fim do quantum, nunca no meio dele, nem mesmo quando chega um processo mais prioritário. No fim do quantum:

1. O processo atual, se não finalizou, volta a READY e entra no fim da fila.
2. O escalonador seleciona o processo da fila com o menor número de prioridade. Em empate, vence o que está há mais tempo na fila.

Consequências: processos de mesma prioridade se alternam como em RR; se o processo atual for o único de melhor prioridade, ele é selecionado novamente (registra-se *Quantum esgotado* e *Executando* para o mesmo PID, sem contar troca de contexto).

A fila de prontos pode ser implementada como lista ou fila circular protegida por mutex, com inserção no fim, remoção do início e remoção de elemento arbitrário.

2.6 Versão multiprocessador

O arquivo de entrada define o número de CPUs (1 a 8) e a arquitetura de fila. Um processo executa em no máximo uma CPU por vez. Cada CPU aplica a política de forma independente (os quanta de CPUs diferentes não são sincronizados: cada um começa quando o processo é despachado naquela CPU).

FILA GLOBAL: existe uma única fila de prontos, protegida por mutex. Qualquer CPU livre retira dela o próximo processo segundo a política. Ao fim do quantum, a CPU reinsere o processo e seleciona o próximo na mesma seção crítica, antes que CPUs ociosas sejam notificadas (isso evita que uma CPU ociosa "roube" o processo recém-preemptado).

FILA POR_CPU: cada CPU tem sua fila local e seu mutex.

- *Colocação:* um processo que chega vai para a CPU com menor carga (processos na fila local + processo em execução). Empate: menor índice.
- *Migração:* quando uma CPU precisa selecionar um processo e sua fila local está vazia, ela rouba o último elemento da fila local mais longa (empate: menor índice), desde que essa fila tenha ao menos um processo. CPUs ociosas repetem a tentativa sempre que algum processo é inserido em qualquer fila.
- Cada migração é registrada no log.

Na versão monoprocessador, CPUS vale 1 e FILA é ignorado.

2.7 Sincronização

- Mutex do processo: protege remaining_time, state e demais campos do BCP.
- Mutex da fila de prontos (um por fila): protege inserções e remoções.
- Mutex do log: protege o buffer de log, escrito por várias threads.
- Variáveis de condição: pcb->cv (libera as threads de um processo) e uma condição por escalonador (dorme com fila vazia, acorda com nova inserção).
- Ordem de travamento: defina e documente no README uma ordem global (por exemplo, fila antes de processo) e respeite-a, para evitar deadlocks. Nunca mantenha o mutex da fila enquanto executa usleep.

2.8 Finalização

Cada escalonador termina quando, simultaneamente: (i) todas as filas estão vazias, (ii) generator_done está ativa e (iii) nenhuma CPU está executando um processo. Em seguida, a

thread principal faz pthread_join em todas as threads, grava o log e as métricas em log_execucao_minikernel.txt, libera toda a memória e encerra sem imprimir nada na tela.

2.9 Estruturas

```
typedef enum { READY, RUNNING, FINISHED } ProcessState;

typedef struct {
int pid;
int process_len; /* duração total (ms) */
int remaining_time; /* tempo restante (ms) */
int priority; /* 1 (mais alta) a 5 */
int num_threads;
int start_time; /* chegada (ms) */
int cpu_id; /* CPU atual; -1 se nenhuma */
int first_dispatch; /* instante do 1º despacho; -1 se ainda não */
int finish_time; /* instante de finalização; -1 se ainda não */
ProcessState state;
pthread_mutex_t mutex;
pthread_cond_t cv;
pthread_t *thread_ids;
} PCB;

typedef struct {
PCB *pcb;
int thread_index;
} TCB;
```

O BCP é o núcleo da simulação. O pid, process_len, priority, num_threads e start_time são definidos na leitura e não mudam. remaining_time é decrementado pelas threads e exige acesso atômico. state controla o fluxo das threads: READY para RUNNING (escalonado), RUNNING para FINISHED (remaining_time <= 0) e RUNNING para READY (fim de quantum em RR e Prioridade). thread_ids é usado nos pthread_join finais. Os campos cpu_id, first_dispatch e finish_time alimentam o log e as métricas.

O TCB é alocado dinamicamente para cada thread e passado como argumento à função da thread, que assim sabe a qual processo pertence e qual é o seu índice local.

Outras variáveis globais sugeridas: pcb_list, runqueue (uma por fila), generator_done, quantum, scheduler_type, num_cpus, queue_mode e log_buffer. Use variáveis globais somente quando estritamente necessário.

3. Sua Tarefa

Implementar o Mini-Kernel descrito, em duas versões (alvos make monoprocessador e make multiprocessador), que:

1. Leem o arquivo de entrada (seção 4.1)
2. Criam os processos e suas threads respeitando os tempos de chegada
3. Escalonam segundo a política lida (FCFS, RR ou Prioridade Preemptiva)
4. Mantêm a(s) fila(s) de prontos e, no modo POR_CPU, realizam colocação e migração
5. Registram o log de eventos e calculam as métricas
6. Sincronizam corretamente threads e escalonadores, encerram todas as threads e liberam a memória

4. Entrada e Saída

4.1 Entrada

O programa recebe como único argumento o nome de um arquivo de texto, com o seguinte formato (os cabeçalhos aparecem sempre nesta ordem; os arquivos de teste são sempre válidos):

```
CPUS <numero de CPUs, 1 a 8>
QUANTUM <ms, multiplo de 100>
POLITICA <FCFS | RR | PRIORIDADE>
FILA <GLOBAL | POR_CPU>
PROCESSOS <n>
<PID> <CHEGADA_ms> <DURACAO_ms> <PRIORIDADE> <NUM_THREADS>
... (n linhas, uma por processo)
```

- Os processos não têm ordem garantida no arquivo (não assuma que estão ordenados por chegada ou PID).
- NUM_THREADS varia de 1 a 4.
- QUANTUM é ignorado em FCFS. Na versão monoprocessador, CPUS vale 1 e FILA é ignorado.

Exemplo (monoprocessador):

```
CPUS 1
QUANTUM 500
POLITICA FCFS
FILA GLOBAL
PROCESSOS 3
1 0 1000 3 2
2 200 2000 1 1
3 100 1500 2 1
```

4.2 Saída: log de eventos

O arquivo log_execucao_minikernel.txt (este é o único nome, para qualquer política ou versão) contém os eventos em ordem cronológica, um por linha. <POL> é FCFS, RR ou PRIORIDADE, e t é o tempo em ms desde T0.

```
[t=<t>ms] [<POL>] Chegada PID <p>
[t=<t>ms] [<POL>] [CPU<k>] Executando PID <p>
[t=<t>ms] [<POL>] [CPU<k>] Quantum esgotado PID <p> (somente RR e PRIORIDADE)
[t=<t>ms] [<POL>] [CPU<k>] PID <p> finalizado
[t=<t>ms] [<POL>] [CPU<k>] Migracao PID <p> CPU<j> -> CPU<k> (somente FILA POR_CPU)
[t=<t>ms] [<POL>] Escalonador terminou execução de todos processos
```

Na versão monoprocessador, a única CPU é CPU0.

4.3 Saída: relatório de métricas

Após a última linha do log, o mesmo arquivo recebe o bloco de métricas (PIDs em ordem crescente, valores médios com duas casas decimais), conforme os exemplos abaixo. Definições:

- Espera = turnaround − duração
- Turnaround = instante de finalização − chegada
- Resposta = instante do primeiro despacho − chegada
- Makespan = instante da última finalização
- Troca de contexto = despacho de um PID diferente do último executado naquela CPU (o primeiro despacho de cada CPU conta)
- Utilização da CPU = tempo efetivamente executando processos ÷ makespan (a retenção ociosa do RR/Prioridade não conta como execução)
- Throughput = número de processos ÷ makespan em segundos

4.4 Exemplo: monoprocessador, FCFS

Para a entrada da seção 4.1, a saída esperada é:

```
[t=0ms] [FCFS] Chegada PID 1
[t=0ms] [FCFS] [CPU0] Executando PID 1
[t=100ms] [FCFS] Chegada PID 3
[t=200ms] [FCFS] Chegada PID 2
[t=1000ms] [FCFS] [CPU0] PID 1 finalizado
[t=1000ms] [FCFS] [CPU0] Executando PID 3
[t=2500ms] [FCFS] [CPU0] PID 3 finalizado
[t=2500ms] [FCFS] [CPU0] Executando PID 2
[t=4500ms] [FCFS] [CPU0] PID 2 finalizado
[t=4500ms] [FCFS] Escalonador terminou execução de todos processos
=== METRICAS ===
PID 1 | espera=0ms | turnaround=1000ms | resposta=0ms
PID 2 | espera=2300ms | turnaround=4300ms | resposta=2300ms
PID 3 | espera=900ms | turnaround=2400ms | resposta=900ms
Espera media: 1066.67 ms
Turnaround medio: 2566.67 ms
Resposta media: 1066.67 ms
```

```
Trocas de contexto: 3
Utilizacao CPU0: 100.00%
Throughput: 0.67 processos/s
Makespan: 4500 ms
```

4.5 Exemplo: multiprocessador (2 CPUs, FCFS, fila global)

Mesma carga, com o cabeçalho CPUS 2, QUANTUM 500, POLITICA FCFS, FILA GLOBAL:

```
[t=0ms] [FCFS] Chegada PID 1
[t=0ms] [FCFS] [CPU0] Executando PID 1
[t=100ms] [FCFS] Chegada PID 3
[t=100ms] [FCFS] [CPU1] Executando PID 3
[t=200ms] [FCFS] Chegada PID 2
[t=1000ms] [FCFS] [CPU0] PID 1 finalizado
[t=1000ms] [FCFS] [CPU0] Executando PID 2
[t=1600ms] [FCFS] [CPU1] PID 3 finalizado
[t=3000ms] [FCFS] [CPU0] PID 2 finalizado
[t=3000ms] [FCFS] Escalonador terminou execução de todos processos
=== METRICAS ===
PID 1 | espera=0ms | turnaround=1000ms | resposta=0ms
PID 2 | espera=800ms | turnaround=2800ms | resposta=800ms
PID 3 | espera=0ms | turnaround=1500ms | resposta=0ms
Espera media: 266.67 ms
Turnaround medio: 1766.67 ms
Resposta media: 266.67 ms
Trocas de contexto: 3
Utilizacao CPU0: 100.00%
Utilizacao CPU1: 50.00%
Throughput: 1.00 processos/s
Makespan: 3000 ms
```

4.6 Comparação com a saída esperada

Como a simulação usa usleep e threads reais, a comparação automática tolera: (i) desvio de até 150 ms nos instantes dos eventos; (ii) permutação entre eventos cujos instantes diferem menos que 150 ms (por exemplo, eventos simultâneos em CPUs diferentes); (iii) desvio de até 10% nas métricas. A ordem dos eventos de uma mesma CPU, os PIDs, as CPUs e a contagem de eventos devem coincidir.

4.7 Execução

O professor executará:

tar -xzvf <nome_arquivo>.tar.gz
make <monoprocessador | multiprocessador>
./trabSO <nome_arquivo_entrada>

Siga esse padrão à risca. O programa não deve solicitar nenhuma entrada nem imprimir nada na tela. Exemplo, para o arquivo 2004209608.tar.gz e a entrada entrada.txt: make monoprocessador seguido de ./trabSO entrada.txt.

5. Detalhes de Implementação

- Linguagem C, na versão disponível nos computadores do LabGrad (Ubuntu). O código deve ser portável (Linux/Unix). Usuários de Windows: testem no Linux (VM, WSL ou LabGrad) antes de entregar.
- Compilação obrigatória com make, gerando o executável trabSO. Compile com -pthread e -Wall.
- Verifique vazamentos com valgrind durante todo o desenvolvimento, e use valgrind --tool=helgrind para detectar condições de corrida e problemas de ordem de travamento. O ideal é zero erros.
- O uso de pthread, mutexes e variáveis de condição é obrigatório. Trabalhe com tempo em milissegundos usando usleep, gettimeofday e similares.
- Cuidado com condições de corrida, deadlocks e *lost wakeups* (sempre teste o predicado em um while ao usar pthread_cond_wait).

6. README e Análise Comparativa

O repositório deve conter um README com:

1. Decisões de projeto: módulos, estruturas, ordem de travamento dos mutexes e como cada regra da seção 2 foi implementada.
2. Análise comparativa, usando a carga carga_analise.txt (disponibilizada no Classroom, com cerca de 10 processos), executada com:
    - as três políticas na versão monoprocessador;
    - a versão multiprocessador com 2 e 4 CPUs, nos modos GLOBAL e POR_CPU;
    - RR com quantum de 100, 500 e 1000 ms.
3. Uma tabela com as métricas obtidas e uma discussão que responda: qual política minimizou a espera média e por quê; o que acontece com os processos de baixa prioridade; como o quantum afeta resposta e trocas de contexto; e o que muda entre fila global e filas por CPU (utilização, migrações, equilíbrio de carga).

7. Regras para o Desenvolvimento

- Data de entrega: 04/12/2026. Não serão aceitos trabalhos após o prazo.
- Grupo: duplas.
- Entrega: pela atividade do Classroom. A entrega por repositório privado no GitHub, compartilhado com o professor, é encorajada, e o README serve para documentar a modelagem. A submissão deve incluir todos os fontes e o Makefile. Apenas um integrante envia, e o nome do arquivo deve conter as matrículas de todos, separadas por hífen, no formato “TP-SO-2026-2-matricula1-matricula2”. O template deve ser rigorosamente seguido para a entrega.

- Recomendações: modularize, mantenha estilo consistente, comente extensivamente e não deixe para a última hora.

8. Avaliação

O trabalho vale 10 pontos:

Item	Pontos
Monoprocessador (FCFS, RR, Prioridade)	3,0
Multiprocessador com FILA GLOBAL	2,0
Multiprocessador com FILA POR_CPU e migração	1,5
Relatório de métricas	1,0
README com análise comparativa	1,5
Organização, modularização e comentários	1,0

- Serão disponibilizados casos de teste públicos com as saídas esperadas do modo monoprocessador. As saídas dos casos multiprocessador serão apenas parcialmente disponibilizadas. Há casos ocultos, incluindo um caso de estresse (dezenas de processos, até 4 threads por processo, 8 CPUs) para detectar condições de corrida.
- Erros de compilação: nota zero.
- O uso de goto: nota zero.
- Segmentation fault em qualquer caso de teste do Classroom: penalização severa. Condições de corrida e deadlocks: penalização severa.
- Vazamento de memória: desconto na nota.
- Código confuso e sem comentários: desconto na nota.
- Detectada cópia (entre alunos ou da Internet), todos os envolvidos recebem nota zero, com direito amplo de defesa conforme as resoluções da UFES. A critério do professor, poderão ser realizadas entrevistas, e a nota dependerá do desempenho nelas.

9. Bônus (opcional, até 1,0 ponto no total, limitado à nota máxima de 10)

- Afinidade de CPU (até 0,5): em caso de empate na seleção, preferir o processo cuja última CPU seja a que está livre. Meça e discuta no README o impacto no número de migrações e nas métricas.